■ Python File Handling

Complete Guide: Read · Write · Append · Clear · All Methods with Real Examples

This guide covers **every file operation** in Python — from opening and reading files to writing, appending, clearing, and working with binary files. Each section includes **when to use it** and **practical examples**.

■ Contents

1. Opening & Closing Files — open() and with
2. File Modes Cheat Sheet
3. Reading Files — read(), readline(), readlines()
4. Writing Files — write(), writelines()
5. Appending to Files — mode 'a'
6. Clearing / Truncating a File
7. Working with Binary Files
8. Checking File Existence & Info
9. Common Patterns & Real-World Examples
10. Quick Reference Table

1. Opening & Closing Files

Every file operation in Python starts with **open()**. Always use the **with** statement — it automatically closes the file even if an error occurs.

```
# ■ Best practice — file closes automatically
with open("data.txt", "r") as f:
    content = f.read()

# ■ Manual close — risky if error occurs
f = open("data.txt", "r")
content = f.read()
f.close()
```

■ **Tip:** Always use **with open(...) as f:** — it's safer and cleaner than calling f.close() manually.

2. File Modes Cheat Sheet

Mode	Meaning	File must exist?	Pointer starts at
"r"	Read only	Yes — error if not	Beginning
"w"	Write (overwrite)	No — creates new	Beginning (erases content)
"a"	Append	No — creates new	End of file
"x"	Exclusive create	No — error if exists	Beginning
"r+"	Read + Write	Yes	Beginning
"w+"	Write + Read	No — creates new	Beginning (erases content)
"a+"	Append + Read	No — creates new	End for write, beginning for read
"rb"	Read binary	Yes	Beginning
"wb"	Write binary	No — creates new	Beginning

3. Reading Files

3a. read() — Read entire file at once

Use when: You need the full content as one string (small files, config files, JSON, etc.).

```
with open("story.txt", "r") as f:
    content = f.read()          # entire file as one string
    print(content)

# Read only first 100 characters
with open("story.txt", "r") as f:
    chunk = f.read(100)
    print(chunk)
```

3b. readline() — Read one line at a time

Use when: Processing a file line by line while keeping low memory usage.

```
with open("data.txt", "r") as f:
    line = f.readline()         # reads first line (includes \n)
    while line:
        print(line.strip())     # strip() removes trailing newline
        line = f.readline()
```

3c. readlines() — Read all lines into a list

Use when: You need random access to lines by index, or want to iterate multiple times.

```
with open("students.txt", "r") as f:
    lines = f.readlines()       # ["Alice\n", "Bob\n", "Carol\n"]

print(lines[0].strip())         # Alice
print(len(lines))               # number of lines

# Iterate
for line in lines:
    print(line.strip())
```

3d. Iterating directly (most Pythonic)

Use when: Reading a large file line by line without loading it all into memory.

```
with open("big_log.txt", "r") as f:
    for line in f:              # memory-efficient — reads one line at a time
        if "ERROR" in line:
            print(line.strip())
```

■ **Tip:** For large files (logs, CSVs), always iterate directly over the file object instead of readlines() to save memory.

3e. tell() and seek() — Control file pointer

Use when: You need to read a file multiple times or jump to a specific position.

```
with open("data.txt", "r") as f:
    print(f.tell())             # 0 — pointer at start
    f.read(10)                  # read 10 chars
    print(f.tell())             # 10
```

```
f.seek(0)                  # go back to beginning
f.seek(5)                  # jump to position 5
f.seek(0, 2)               # jump to end of file
```

4. Writing Files

■■ **Warning:** Mode 'w' always **erases the file first** before writing. Use 'a' to keep existing content.

4a. write() — Write a string

Use when: Saving output, creating config files, generating reports.

```
with open("output.txt", "w") as f:
    f.write("Hello, World!\n")  # \n = newline character
    f.write("Second line\n")

# Writing variables
name = "Alice"
score = 95
with open("result.txt", "w") as f:
    f.write(f"Student: {name}\n")
    f.write(f"Score: {score}\n")
```

4b. writelines() — Write a list of strings

Use when: You have a list of lines to write all at once.

```
students = ["Alice\n", "Bob\n", "Carol\n"]

with open("students.txt", "w") as f:
    f.writelines(students)       # writes each item in the list

# Note: writelines() does NOT add newlines automatically!
names = ["Alice", "Bob", "Carol"]
with open("names.txt", "w") as f:
    f.writelines(name + "\n" for name in names)  # add \n yourself
```

■ ***Note:*** *writelines() does **not** add newlines between items. You must include **\n** in each string yourself.*

4c. print() to a file

Use when: You want print's automatic newline + formatting but writing to a file.

```
with open("log.txt", "w") as f:
    print("Server started", file=f)          # adds \n automatically
    print("Port:", 8080, file=f)             # spaces between args
    print("Status:", "OK", sep="=", file=f)  # Status=OK
```

5. Appending to Files

Use 'a' mode when you want to **add content to the end** without erasing what's already there.

```
# First run — creates file and writes
with open("log.txt", "a") as f:
    f.write("2024-01-10: App started\n")

# Second run — adds to existing content
with open("log.txt", "a") as f:
    f.write("2024-01-10: User logged in\n")
```

```
# Result in log.txt:
# 2024-01-10: App started
# 2024-01-10: User logged in
```

Real-world example — logging errors:

```
import datetime

def log_error(message):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("errors.log", "a") as f:
        f.write(f"[{timestamp}] ERROR: {message}\n")

log_error("Database connection failed")
log_error("Timeout on API call")
```

■ **Tip:** Use **'a'** for log files, history files, or any file where you want to keep adding new data over time.

6. Clearing / Truncating a File

There are several ways to clear a file in Python. Here are all of them:

Method 1: Open with 'w' mode (simplest)

Use when: You want to erase everything and start fresh.

```
# Opening with "w" automatically erases the file
with open("data.txt", "w") as f:
    pass                        # do nothing — file is now empty

# Or just open and close immediately
open("data.txt", "w").close()
```

Method 2: truncate(0) — Clear without closing

Use when: You need to clear the file while keeping it open, e.g. while reading then writing.

```
with open("data.txt", "r+") as f:
    content = f.read()          # read first
    f.seek(0)                   # go back to start
    f.truncate(0)               # clear all content
    f.write("Fresh content\n") # now write new content
```

■■ **Warning:** After truncate(0), always call **f.seek(0)** before writing, otherwise the pointer may be at a wrong position.

Method 3: truncate(size) — Keep only first N bytes

Use when: You want to cut a file to a specific size.

```
with open("data.txt", "r+") as f:
    f.truncate(50)              # keep only first 50 bytes

# Or using os module
import os
os.truncate("data.txt", 0)     # clear the file
os.truncate("data.txt", 100)   # keep first 100 bytes
```

Method 4: Clear and immediately write new content

Use when: You want to replace all content in one step.

```
# Most common real-world pattern
new_data = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("data.txt", "w") as f:   # "w" clears then writes
    f.writelines(new_data)
```

7. Working with Binary Files

Use binary mode (**'rb'**, **'wb'**) for images, PDFs, audio, video, or any non-text file.

```
# Copy an image file
with open("photo.jpg", "rb") as src:
    data = src.read()

with open("photo_copy.jpg", "wb") as dst:
    dst.write(data)

# Copy in chunks (for large files)
with open("video.mp4", "rb") as src, open("copy.mp4", "wb") as dst:
    while chunk := src.read(4096):   # read 4KB at a time
        dst.write(chunk)
```

■ **Tip:** Always use binary mode ('rb'/'wb') for files like images, ZIP archives, PDFs, and audio files.

8. Checking File Existence & Info

```
import os
from pathlib import Path

# Check if file exists
if os.path.exists("data.txt"):
    print("File found!")

# Modern way using pathlib
path = Path("data.txt")
if path.exists():
    print("File found!")

# Get file size
size = os.path.getsize("data.txt")     # in bytes

# Get file name and extension
print(os.path.basename("folder/data.txt"))  # data.txt
print(os.path.splitext("data.txt"))         # ("data", ".txt")

# Delete a file safely
if os.path.exists("old.txt"):
    os.remove("old.txt")

# Rename a file
os.rename("old_name.txt", "new_name.txt")
```

9. Common Patterns & Real-World Examples

■ Count lines, words, and characters in a file

```
with open("essay.txt", "r") as f:
    content = f.read()

lines = content.splitlines()
words = content.split()
chars = len(content)

print(f"Lines: {len(lines)}")
print(f"Words: {len(words)}")
print(f"Characters: {chars}")
```

■ Find and replace text in a file

```
with open("template.txt", "r") as f:
    content = f.read()

content = content.replace("{{NAME}}", "Alice")
content = content.replace("{{DATE}}", "2024-01-10")

with open("output.txt", "w") as f:
    f.write(content)
```

■ Read a CSV file manually

```
with open("students.csv", "r") as f:
    header = f.readline().strip().split(",")  # first line = headers
    for line in f:
        values = line.strip().split(",")
        row = dict(zip(header, values))
        print(row)   # {"name": "Alice", "score": "95"}
```

■ Write a list of dicts to CSV

```
data = [
    {"name": "Alice", "score": 95},
    {"name": "Bob",   "score": 87},
]

with open("results.csv", "w") as f:
    f.write("name,score\n")         # header
    for row in data:
        f.write(f"{row['name']},{row['score']}\n")
```

■ Safe file write (write to temp, then rename)

Use when: You want to avoid data corruption if the program crashes mid-write.

```
import os

# Write to a temp file first
with open("data.txt.tmp", "w") as f:
    f.write("Important data\n")
```

```
# Only replace original when write is complete
os.replace("data.txt.tmp", "data.txt")
```

10. Quick Reference Table

Operation	Method / Mode	When to Use
Read all	"r" + f.read()	Small files, config, JSON
Read one line	"r" + f.readline()	Line-by-line processing
Read all lines	"r" + f.readlines()	Need index access to lines
Iterate lines	for line in f:	Large files — memory efficient
Write (overwrite)	"w" + f.write()	Create/replace a file
Write list	"w" + f.writelines()	Writing many lines at once
Append	"a" + f.write()	Log files, adding history
Clear file	"w" mode or truncate(0)	Reset a file to empty
Read + Write	"r+" mode	Modify file in-place
Binary read	"rb" + f.read()	Images, PDFs, audio, video
Binary write	"wb" + f.write()	Save binary data / copy files
File pointer	f.tell() / f.seek()	Jump to position in file
Exclusive create	"x" mode	Create file, error if exists

Key Rules to Remember:

1. Always use **with open(...)** — never forget to close files.
2. Mode **'w'** erases the file. Mode **'a'** keeps existing content.
3. Use **binary mode** ('rb'/'wb') for non-text files.
4. For large files, iterate directly: **for line in f** — not readlines().
5. Use **truncate(0)** with seek(0) when you need to clear an open file.